

AI vs Real Image Detection using EfficientNet-B0

A deep learning-based image classification project that detects whether an image is **AI-generated** or **real** using a pretrained **EfficientNet-B0** model.

The project uses transfer learning with PyTorch and evaluates the model using multiple classification metrics, including Accuracy, Precision, Recall, F1-Score, and ROC-AUC. **Grad-CAM** is also used to provide visual explanations of the model's predictions.

Project Overview

The rapid growth of generative AI has made it increasingly difficult to distinguish AI-generated images from real photographs.

This project investigates the use of deep learning to perform **binary image classification**:

- **Class 0:** AI-Generated
- **Class 1:** Real

A pretrained EfficientNet-B0 architecture is fine-tuned on an AI-vs-Real image dataset. The trained model is then evaluated on an unseen test set.

The project also uses **Grad-CAM (Gradient-weighted Class Activation Mapping)** to visualize the image regions that contribute to the model's prediction.

Objectives

The main objectives of this project are:

1. Build a binary image classification model for AI-generated and real images.
 2. Use transfer learning with a pretrained EfficientNet-B0 architecture.
 3. Apply image preprocessing and augmentation techniques.
 4. Divide the dataset into training, validation, and testing subsets.
 5. Train the model using an appropriate loss function and optimizer.
 6. Use early stopping to reduce unnecessary training.
 7. Apply learning-rate scheduling.
 8. Evaluate the model using multiple classification metrics.
 9. Visualize classification performance using a confusion matrix and ROC curve.
 10. Use Grad-CAM to improve model interpretability.
-

Model Architecture

The project uses:

EfficientNet-B0

The model is loaded using the `timm` library with pretrained weights.

```
timm.create_model(  
    'efficientnet_b0',  
    pretrained=True,  
    num_classes=2  
)
```

The final classifier is configured for two classes:

```
0 → AI-Generated  
1 → Real
```

Model Parameters

The configured model contains approximately:

- **Total parameters:** 4,010,110
- **Trainable parameters:** 4,010,110

Dataset

The project uses the **AI-vs-Real** dataset hosted on Hugging Face:

```
Parveshiiii/AI-vs-Real
```

The dataset loaded by the notebook contains:

```
13,999 images
```

The available features are:

```
image
binary_label
```

Dataset Split

The dataset is divided into:

Split	Percentage	Images
Training	70%	9,799
Validation	15%	2,099
Testing	15%	2,101
Total	100%	13,999

A fixed random seed of `42` is used when creating the splits.

Data Preprocessing

All images are resized to:

```
224 × 224
```

Training Augmentation

The training pipeline applies:

- Resize
- Random horizontal flip
- Random rotation
- Color jitter
- Random affine translation
- Conversion to tensor
- ImageNet normalization

The training transformation includes:

```
transforms.Resize((224, 224))
transforms.RandomHorizontalFlip(p=0.5)
transforms.RandomRotation(15)
transforms.ColorJitter(
    brightness=0.2,
```

```
        contrast=0.2,  
        saturation=0.2,  
        hue=0.1  
    )  
    transforms.RandomAffine(  
        degrees=0,  
        translate=(0.1, 0.1)  
    )
```

Validation and Test Preprocessing

Validation and test images use:

Resize → ToTensor → ImageNet Normalization

No random augmentation is applied to validation or test images.

Training Configuration

Parameter	Value
Model	EfficientNet-B0
Pretrained	Yes
Image Size	224 × 224
Number of Classes	2
Batch Size	32
Maximum Epochs	20
Learning Rate	0.0001
Weight Decay	0.00001
Optimizer	Adam
Loss Function	CrossEntropyLoss
Random Seed	42
Early Stopping Patience	3
LR Scheduler	ReduceLROnPlateau
LR Reduction Factor	0.5

Parameter	Value
LR Scheduler Patience	3

Training Process

The training process follows the standard deep learning workflow:

```
Input Images
↓
Image Preprocessing
↓
EfficientNet-B0
↓
Forward Pass
↓
Cross-Entropy Loss
↓
Backpropagation
↓
Adam Optimizer
↓
Validation
↓
Learning Rate Scheduling
↓
Early Stopping
↓
Best Model
```

During training, the model tracks:

- Training loss
- Training accuracy
- Validation loss
- Validation accuracy
- Learning rate

The model with the lowest validation loss is saved as the best model.

Early Stopping

Early stopping is implemented with a patience of:

3 epochs

If the validation loss does not improve for three consecutive epochs, training is stopped.

This helps prevent unnecessary training once the model stops improving on the validation set.



Model Evaluation

The trained model is evaluated using the separate test set containing:

2,101 images

The following metrics are calculated:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC
- Test Loss

A classification report is also generated for both classes.



Test Results

The final test results obtained from the notebook are:

Metric	Score
Accuracy	98.62%
Precision	99.38%
Recall	98.83%
F1-Score	99.10%
ROC-AUC	99.84%
Test Loss	0.0517

Classification Report

Class	Precision	Recall	F1-Score	Support
AI-Generated	0.96	0.98	0.97	483
Real	0.99	0.99	0.99	1,618
Overall Accuracy			0.99	2,101

The test set results indicate that the trained model achieved high classification performance on this dataset.

Note: These results represent performance on the dataset and split used in this project. They should not automatically be interpreted as equivalent performance on images generated by unseen AI tools or on real-world images outside the dataset.



Evaluation Visualizations

The notebook generates several visualizations to analyze the model.

Training History

The training history includes:

- Training vs validation loss
- Training vs validation accuracy
- Learning-rate schedule

Generated file:

```
results/training_history.png
```

Confusion Matrix and ROC Curve

The project generates:

- Confusion matrix
- ROC curve
- ROC-AUC value

Generated file:

```
results/evaluation_metrics.png
```

Grad-CAM Visualization

Grad-CAM is used to visualize regions of an image that contribute to the model's prediction.

Generated file:

```
results/gradcam_visualization.png
```

Model Explainability with Grad-CAM

High classification accuracy alone does not explain **why** a model makes a prediction.

To improve interpretability, this project uses **Grad-CAM**.

The implementation uses the EfficientNet-B0 convolutional head:

```
target_layers = [model.model.conv_head]
```

Grad-CAM produces a heatmap showing the areas of the input image that contributed to the model's selected prediction.

The visualization compares:

```
Original Image
      +
Grad-CAM Heatmap
```

This provides a visual method for inspecting the model's decision-making process.

Project Structure

A recommended GitHub repository structure is:

```
AI-vs-Real-Image-Detection/
|
|— README.md
|
|— AI_vs_Real_Detection.ipynb
|
```

```
├── models/
│   └── best_model.pth
├── results/
│   ├── sample_images.png
│   ├── training_history.png
│   ├── evaluation_metrics.png
│   ├── gradcam_visualization.png
│   └── test_results.csv
└── requirements.txt
```

The exact files included in the repository may depend on which generated artifacts you choose to commit.

Technologies Used

Programming Language

- Python

Deep Learning

- PyTorch
- Torchvision
- timm

Dataset

- Hugging Face Datasets

Machine Learning

- Scikit-learn

Image Processing

- Pillow
- OpenCV

Visualization

- Matplotlib
- Seaborn

Explainable AI

- Grad-CAM

Development Environment

- Google Colab
-

Installation

Install the required Python packages using:

```
pip install torch torchvision timm datasets pillow numpy pandas matplotlib
seaborn tqdm scikit-learn grad-cam
```

Or create a `requirements.txt` file containing:

```
torch
torchvision
timm
datasets
pillow
numpy
pandas
matplotlib
seaborn
tqdm
scikit-learn
grad-cam
```

Then install them with:

```
pip install -r requirements.txt
```

Running the Project

Option 1 — Google Colab

The notebook was developed to run in Google Colab.

1. Open the notebook in Google Colab.
2. Install the required packages.
3. Enable the required runtime.
4. Run the notebook cells sequentially.

5. The dataset will be loaded from Hugging Face.
6. The model will be trained.
7. Evaluation results and visualizations will be generated.

Option 2 — Local Environment

The notebook can also be adapted to run locally.

You will need:

- Python
- PyTorch
- torchvision
- timm
- Hugging Face Datasets
- CUDA-compatible GPU (recommended for faster training)

The original notebook uses Google Drive paths for saving the model and results, so these paths should be changed when running outside Google Colab.

For example:

```
PROJECT_DIR = "./AI-vs-Real-Image-Detection"
```

instead of the original Google Drive path.

Saved Model

The best-performing model is saved as:

```
models/best_model.pth
```

The saved checkpoint contains:

- Epoch
 - Model state dictionary
 - Optimizer state dictionary
 - Training loss
 - Validation loss
 - Training accuracy
 - Validation accuracy
-

Generated Results

The notebook generates the following files:

```
results/  
├─ training_history.png  
├─ evaluation_metrics.png  
├─ gradcam_visualization.png  
└─ test_results.csv
```

The CSV file contains:

Metric
Value

for:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC
- Test Loss

Limitations

Although the model achieves strong performance on the test set, several limitations should be considered.

Dataset Dependency

The model's performance depends on the characteristics and distribution of the dataset used during training.

Generalization

High test accuracy does not guarantee the same performance on:

- Images generated by new AI models
- Images generated by unseen tools
- Heavily edited images
- Compressed images
- Screenshots
- Social-media images

- Real-world photographs outside the dataset

Dataset Imbalance

The test set contains:

AI-Generated: 483
Real: 1,618

Therefore, accuracy should be considered alongside Precision, Recall, F1-Score, and ROC-AUC.

Computational Requirements

Training the model can be computationally expensive, particularly without GPU acceleration.

Future Improvements

Possible future improvements include:

- Testing against images generated by previously unseen AI generators.
- Evaluating cross-generator generalization.
- Increasing dataset diversity.
- Testing additional EfficientNet variants.
- Comparing EfficientNet with other CNN architectures.
- Exploring Vision Transformers.
- Performing more extensive hyperparameter tuning.
- Using additional explainability methods.
- Evaluating robustness against image compression and resizing.
- Building a web interface for uploading an image and obtaining a prediction.
- Deploying the trained model as an API.

Research Relevance

This project demonstrates the application of deep learning and explainable AI to the problem of distinguishing AI-generated images from real images.

The project combines:

Computer Vision
+
Transfer Learning
+

Image Classification
+
Model Evaluation
+
Explainable AI

It can therefore serve as a practical demonstration of deep learning model development, evaluation, and interpretability.

Author

Ashish Thapa

MSc IT Student
Islington College, Nepal

License

If this project uses a dataset or pretrained model with its own licensing terms, their respective licenses and usage conditions should be followed.

The dataset used in this project is hosted on Hugging Face:

Parveshiiii/AI-vs-Real

Please check the original dataset repository for its specific license and usage requirements before redistributing the dataset.

Acknowledgements

- PyTorch
 - Torchvision
 - Hugging Face Datasets
 - timm
 - Scikit-learn
 - Grad-CAM
-

Project Summary

AI vs Real Image Detection using EfficientNet-B0 is a binary image classification project that uses transfer learning to distinguish AI-generated images from real images.

The final model achieved the following results on the project's test set:

Accuracy	: 98.62%
Precision	: 99.38%
Recall	: 98.83%
F1-Score	: 99.10%
ROC-AUC	: 99.84%

The project also incorporates **Grad-CAM** to provide visual explanations of model predictions.
